

Cyber Security & Ethical Hacking

Laboratorio giorno 4 – Cisco CyberOps

Esercizio: Lavorare con File di Testo nella CLI

BONUS: Prendere Familiarità con la Shell Linux

Nik Madonna

3 settembre 2026

Indice

1	Parte 1: Editor di Testo Grafici (SciTE)	2
2	Parte 2: Editor di Testo da Riga di Comando (nano)	2
3	Parte 3: Lavorare con i File di Configurazione	2
4	BONUS – Parte 1: Basi della Shell	3
5	BONUS – Passo 3: Creare e Cambiare Directory	4
6	BONUS – Passo 4 e 5: Redirigere e Accodare Output	4
7	BONUS – Passo 6: Lavorare con i File Nascosti	5
8	BONUS – Parte 2: Copiare, Eliminare e Spostare File	5

1 Parte 1: Editor di Testo Grafici (SciTE)

Domanda: Sei riuscito a trovare subito `space.txt`?

No. Aprendo la finestra *File > Open* di SciTE con il filtro predefinito, il file `space.txt` non compariva nell'elenco, nonostante SciTE stesse guardando nella directory corretta (`/home/analyst`). Questo accade perché SciTE filtra i file in base a estensioni note, e `.txt` non rientra tra queste per impostazione predefinita. È stato necessario selezionare **All Files (*)** dal menu a discesa per visualizzare e aprire correttamente il file.

Domanda: Perché il prompt non viene mostrato nel terminale?

Quando SciTE viene avviato dalla riga di comando con `scite space.txt`, il processo occupa il terminale in *foreground*: la shell resta in attesa che il processo SciTE termini prima di restituire nuovamente il controllo (e quindi il prompt) all'utente. Il terminale è quindi ancora attivo in *background*, ma non può accettare nuovi comandi finché SciTE non viene chiuso o il processo non viene interrotto (es. con `CTRL+C`).

2 Parte 2: Editor di Testo da Riga di Comando (nano)

Domanda: Quale carattere usa nano per rappresentare che una linea continua oltre i bordi dello schermo?

Il simbolo \$ (dollaro), posizionato alla fine della riga visibile, indica che la linea di testo prosegue oltre il bordo destro dello schermo.

3 Parte 3: Lavorare con i File di Configurazione

Domanda: Anche la finestra del terminale che era già aperta ha cambiato colore da verde a rosso? Spiega.

No. La finestra di terminale già aperta in precedenza ha mantenuto il colore verde. Questo perché il file `.bashrc` viene letto ed eseguito da bash solamente all'avvio di una nuova sessione di shell interattiva. Una sessione già avviata ha già caricato la configurazione precedente in memoria e non rilegge automaticamente il file dopo una modifica; è necessario aprire un nuovo terminale (o ricaricare manualmente il file, ad esempio con `source .bashrc` oppure lanciando una nuova shell con il comando `bash`) affinché le modifiche abbiano effetto.

Domanda: Perché i file di configurazione delle applicazioni utente sono salvati nella directory home dell'utente e non sotto `/etc` con tutti gli altri file di configurazione a livello di sistema?

I file di configurazione collocati nella directory home (come `.bashrc`) riguardano impostazioni personali e specifiche di un singolo utente, che non influenzano gli altri utenti del sistema. Poiché ogni utente ha sempre pieno accesso in scrittura alla propria directory home, può personalizzare liberamente il comportamento delle proprie applicazioni senza necessitare di privilegi elevati. Al contrario, i file sotto `/etc` regolano il comportamento di servizi e applicazioni a livello dell'intero sistema, influenzando tutti gli utenti: per questo motivo sono scrivibili solo dall'utente `root`, così da evitare che un utente non autorizzato alteri il comportamento globale del sistema.

Domanda: A cosa si riferisce il messaggio di errore?

Il messaggio `bind() to 0.0.0.0:8080 failed (98: Address already in use)` indica che la porta TCP 8080 era già occupata da un'altra istanza del processo `nginx`, avviata in un tentativo precedente e ancora attiva. Poiché due processi non possono rimanere in ascolto

simultaneamente sulla stessa porta, il nuovo tentativo di avvio falliva. Il problema è stato risolto terminando il processo `nginx` già in esecuzione con `sudo pkill nginx` prima di rilanciare il servizio con la configurazione aggiornata.

Domanda: Appare la pagina web?

No. Dopo aver eseguito `sudo pkill nginx`, il servizio web viene arrestato e non risponde più alle richieste sulla porta 8080. Ricaricando la pagina `127.0.0.1:8080` nel browser (dopo aver pulito la cronologia/cache), la pagina non viene più visualizzata, confermando che il webserver `nginx` è stato effettivamente spento.

Domanda: Domanda Sfida: Puoi modificare il file `/etc/nginx/custom_server.conf` con SciTE? Descrivi il processo di seguito.

Sì, è possibile modificare il file `/etc/nginx/custom_server.conf` con SciTE, ma solo avviando l'applicazione con privilegi di root, poiché la directory `/etc` non è scrivibile dall'utente standard `analyst`.

Il processo seguito è stato:

1. Aprendo SciTE normalmente dal menu grafico (senza privilegi elevati) e tentando di aprire il file tramite *File > Open*, il file risulta visibile e apribile in lettura (i permessi di lettura sono generalmente concessi a tutti gli utenti). Tentando però di salvare una modifica con *File > Save*, l'operazione fallisce con un errore di permesso negato, poiché il processo SciTE gira con i privilegi dell'utente `analyst`, che non ha diritti di scrittura su `/etc`.
2. Per modificare correttamente il file, è stato invece aperto un terminale ed eseguito il comando:

```
[analyst@secOps ~]$ sudo scite /etc/nginx/custom_server.conf
[sudo] password for analyst:
```

3. Dopo aver inserito la password, SciTE si apre automaticamente con il file `custom_server.conf` già caricato. Questa volta il processo SciTE eredita i privilegi di root (essendo stato lanciato con `sudo`), pertanto la modifica del file e il successivo salvataggio con *File > Save* vanno a buon fine senza errori.

4 BONUS – Parte 1: Basi della Shell

Domanda: Elenca alcune sezioni incluse in una pagina `man`.

Consultando la pagina `man` di `man` stesso (`man man`), alcune delle sezioni principali riscontrate sono: **NAME**, **SYNOPSIS**, **DESCRIPTION** ed **EXAMPLES**. La pagina `man` elenca inoltre, come esempio di sezioni convenzionali, anche *CONFIGURATION*, *OPTIONS*, *EXIT STATUS*, *RETURN VALUE*, *ERRORS*, *ENVIRONMENT*, *FILES*, *VERSIONS*, *STANDARDS*, *NOTES*, *BUGS*, *AUTHORS* e *SEE ALSO*.

Domanda: Qual è la funzione del comando `cp`?

Consultando `man cp`, il comando `cp` viene descritto come: *"copy files and directories"* – ovvero copia file e directory da una sorgente (SOURCE) a una destinazione (DEST), oppure più sorgenti in una directory di destinazione.

Domanda: Quale comando useresti per trovare maggiori informazioni sul comando `pwd`? Qual è la funzione del comando `pwd`?

Per ottenere maggiori informazioni sul comando `pwd` si utilizza `man pwd`. La funzione del comando `pwd` (*print working directory*) è quella di stampare a schermo il percorso completo della directory di lavoro corrente.

5 BONUS – Passo 3: Creare e Cambiare Directory

Domanda: Qual è la directory corrente?

Eseguendo il comando `pwd` subito dopo l'accesso, la directory corrente risulta essere `/home/analyst`, ovvero la directory home dell'utente `analyst`.

Domanda: In quale cartella ti trovi ora?

Dopo l'esecuzione del comando `cd /home/analyst/cyops_folder3`, la directory corrente diventa `/home/analyst/cyops_folder3`, come confermato anche dal prompt che cambia in `[analyst@secOps cyops_folder3]$`.

Domanda: Sfida: Digita il comando `cd ~` e descrivi cosa succede. Perché è successo?

Digitando `cd ~` da una qualsiasi directory, la shell riporta l'utente direttamente nella propria directory home (`/home/analyst`), indipendentemente dalla directory di partenza. Questo accade perché il simbolo tilde (`~`) è una scorciatoia interpretata dalla shell come riferimento diretto alla directory home dell'utente corrente, evitando così di dover digitare il percorso assoluto completo.

Domanda: Cosa succede? (dopo `cd .`)

Non succede alcun cambiamento visibile: la directory corrente rimane invariata. Questo perché il simbolo `.` (punto singolo) rappresenta un riferimento alla directory corrente stessa, quindi eseguire `cd .` equivale a "spostarsi" nella directory in cui ci si trova già.

Domanda: Cosa succede? (dopo `cd ..`)

Il prompt cambia e la directory corrente si sposta di un livello superiore nella gerarchia del filesystem, alla cosiddetta *directory genitore*. Ad esempio, spostandosi da `cyops_folder3` con `cd ..`, si ritorna alla directory home `/home/analyst`.

Domanda: Quale sarebbe la directory corrente se eseguiessi il comando `cd ..` da `[analyst@secOps ~]$`?

Sarebbe `/home`, in quanto la directory home dell'utente (`/home/analyst`) ha come genitore la directory `/home`.

Domanda: Quale sarebbe la directory corrente se eseguiessi il comando `cd ..` da `[analyst@secOps home]$`?

Sarebbe `/` (la directory radice), poiché `/home` ha come genitore la directory radice del filesystem.

Domanda: Quale sarebbe la directory corrente se eseguiessi il comando `cd ..` da `[analyst@secOps /]$`?

Rimarrebbe `/` (la directory radice). La directory radice non ha una directory genitore superiore: eseguire `cd ..` da `/` non produce alcun cambiamento, poiché `/` è il punto più alto della gerarchia del filesystem e fa riferimento a se stessa.

6 BONUS – Passo 4 e 5: Redirigere e Accodare Output

Domanda: È previsto? Spiega. (nessun output mostrato dopo il redirect con `>`)

Sì, è previsto. L'operatore `>` redirige l'output del comando `echo`, che normalmente verrebbe stampato a schermo, verso il file di testo specificato (`some_text_file.txt`) anziché verso il terminale. Di conseguenza, nella finestra del terminale non compare alcun output visibile, perché il testo è stato scritto direttamente nel file.

Domanda: Cosa è successo al file di testo? Spiega.

(dopo il secondo redirect con `>`) Il contenuto precedente del file `some_text_file.txt` è stato completamente sovrascritto dal nuovo messaggio. L'operatore `>` infatti tronca il file di destinazione prima di scrivervi il nuovo output, motivo per cui il comando `cat` mostra solamente il messaggio più recente, senza traccia del contenuto precedente.

Domanda: Cosa è successo al file di testo? Spiega. (dopo l'uso di `>`)

Il nuovo testo è stato aggiunto (*accodato*) alla fine del file, senza cancellare il contenuto già presente. A differenza di `>`, l'operatore `>>` non tronca il file, ma vi appende i nuovi dati mantenendo intatto tutto ciò che era già stato scritto in precedenza. Il comando `cat` mostra infatti sia il messaggio precedente sia quello appena aggiunto.

7 BONUS – Passo 6: Lavorare con i File Nascosti

Domanda: Quanti file vengono visualizzati?

Con il comando `ls -l` nella directory home dell'utente, dopo la creazione delle cartelle `cyops_folder1`, `cyops_folder2` e `cyops_folder3`, vengono visualizzati 10 elementi: il file `capture.pcap` e le directory `cyops_folder1`, `cyops_folder2`, `cyops_folder3`, `Desktop`, `Downloads`, `lab.support.files`, `scripts`, `second_drive` e `yay`. Nessuno di questi elementi risulta essere un file nascosto.

Domanda: Quanti file in più vengono visualizzati rispetto a prima? Spiega.

Aggiungendo l'opzione `-a`, compaiono diversi elementi aggiuntivi rispetto all'output di `ls -l`: le due directory speciali `.` (directory corrente) e `..` (directory genitore), oltre a numerosi file e directory nascosti (i cosiddetti *dotfiles*) legati alla configurazione personale dell'utente, come `.bashrc`, `.bash_history`, `.bash_profile`, `.bash_logout`, `.cache`, `.config`, `.mozilla` e altri. Questi elementi non vengono mostrati con il semplice `ls -l` perché il loro nome inizia con un punto, che li marca come nascosti per convenzione.

Domanda: È possibile nascondere intere directory aggiungendo un punto prima del loro nome? Ci sono directory nascoste nell'output di `ls -la` sopra?

Sì, è possibile nascondere anche intere directory, non solo i singoli file, semplicemente antepo-
nendo un punto al loro nome. Nell'output di `ls -la` sono infatti presenti diverse directory nascoste, come ad esempio `.cache`, `.config`, `.mozilla`, `.gnupg` e `.local`.

Domanda: Fornisci tre esempi di file nascosti mostrati nell'output di `ls -la` sopra.
Tre esempi di file nascosti sono: `.bashrc`, `.bash_history` e `.bash_profile`.

8 BONUS – Parte 2: Copiare, Eliminare e Spostare File

Domanda: Identifica i parametri nel comando `cp` sopra. Quali sono i file sorgente e destinazione? (usa percorsi completi per rappresentare i parametri)

Nel comando `cp some_text_file.txt cyops_folder2/`, i due parametri sono:

- **Sorgente:** `/home/analyst/some_text_file.txt`
- **Destinazione:** `/home/analyst/cyops_folder2/`

Il comando `cp` crea una copia del file sorgente nella posizione di destinazione, lasciando inalterato il file originale, come confermato dal fatto che `some_text_file.txt` risulta presente sia nella directory home sia in `cyops_folder2`.

Domanda: Quale comando hai usato per completare l'attività?

Il comando utilizzato è stato:

```
[analyst@sec0ps ~]$ mv cyops_folder2/some_text_file.txt .
```

Il comando `mv` sposta il file `some_text_file.txt` da `cyops_folder2` alla directory corrente (rappresentata dal punto `.`, ovvero `/home/analyst`), rimuovendolo dalla posizione originale. Diversamente da `cp`, il file non viene duplicato ma spostato: infatti, dopo l'operazione, `cyops_folder2` risulta vuota mentre il file compare nuovamente nella directory `home`.

Domanda: Riflessione: Quali sono i vantaggi dell'utilizzo della riga di comando Linux?

L'utilizzo della riga di comando in Linux offre diversi vantaggi rispetto alle interfacce grafiche:

- Permette di amministrare sistemi da remoto tramite SSH, anche quando non è disponibile alcuna interfaccia grafica (ambienti *headless*);
- Consente di automatizzare attività ripetitive tramite script, riducendo tempi ed errori umani;
- Richiede meno risorse di sistema rispetto a un'interfaccia grafica, risultando più efficiente su server e macchine con risorse limitate;
- Offre un controllo più preciso e granulare sulle operazioni eseguite, con accesso diretto a tutte le opzioni disponibili per ciascun comando (consultabili tramite le pagine `man`);
- È lo standard de facto per l'amministrazione di sistemi Linux/Unix in ambito professionale e per attività di sicurezza informatica.